ChatGPT

Instant Variometer Filter Design: Kalman, Complementary and Stacked Estimation for Minimum-Latency Audio

Executive summary

The central engineering problem in an “instant” variometer is **not measuring pressure quickly**. It is estimating vertical velocity quickly enough to be useful while suppressing pressure noise, aerodynamic artefacts, IMU bias, orientation error and audio chatter. Any conventional strategy that solves noise simply by smoothing pressure creates delay; Bosch explicitly documents that its own barometer IIR filtering reduces bandwidth, and BlueFlyVario's current manual explicitly notes a delay between movement and sensed lift caused by filtering. [1]

The strongest architecture I found in academic work and practical open-source variometers is a **loosely coupled barometer–IMU estimator**:

> **IMU → attitude estimation → Earth-frame vertical acceleration → linear vertical Kalman filter → confidence/audio state machine**

This is essentially the architecture demonstrated by Sabatini and Genovese: a quaternion EKF estimates orientation, acceleration is rotated into the navigation frame and gravity-compensated, and a second estimator fuses that vertical acceleration with barometric altitude. They reported vertical-velocity RMSE of 0.04–0.24 m/s in their laboratory motion experiments, though they did **not** report paraglider onset/release latency. [2] A separate two-stage Kalman/complementary-filter paper by Lee reported mean vertical-position error of 19.7 cm versus 43.4 cm from the raw barometer in its motion-capture experiment. [3] The directly paraglider-focused Brzozowski et al. paper likewise used cascaded Kalman/complementary filtering for hang-glider/paraglider instrumentation. [4]

For our USB-C vario, my **single best approach** is therefore:

ESP32-S3 + IMU at about 200–500 Hz + barometer at about 80–150 Hz + attitude estimator + three-state linear KF `[altitude, vertical velocity, accelerometer bias]` **+ a parallel slower validation estimator + a deliberately interruptible audio state machine.**

The slow estimator should **not be averaged into the fast output**. It should provide confidence, plausibility checking and fallback. Averaging fast and slow estimates would largely put the latency back in.

For the hardware we already own, start with:

ESP32-S3 + MPU6050 at 200 Hz + one MS5611 at 50–100 Hz, recording everything raw with timestamps. The MS5611 is adequate for proving the algorithms; TE specifies conversion times from about 0.54 ms at OSR 256 to 8.22 ms typical at OSR 4096, with noise/resolution improving as conversion time rises. [5]

For the eventual hardware, **BMP581/BMP585 is substantially more attractive for this particular latency problem**. Bosch specifies, for example, roughly 0.30 Pa RMS at ×8 pressure oversampling and 155 Hz continuous ODR, or 0.21 Pa RMS at ×16 and 87 Hz; the highest-resolution 0.08 Pa mode falls to only about 12 Hz. This is almost a perfect illustration of the noise-versus-latency trade: the lowest quoted noise setting is **not** the setting I would use for an instant vario. [6]

The BMP58x's internal IIR should initially be **disabled**. Bosch's own datasheet warns that it reduces signal bandwidth and shows increasingly long step responses as its coefficient rises. [7] Filter pressure in our estimator instead, where we know exactly what latency and state history we are introducing.

The research also confirms your concern about **exit behaviour**. BlueFly's July 2026 manual states that once a lift beep starts it runs for the cadence-determined duration, then its silence interval begins; only at that sequence boundary can beeping end. Its default lift-on threshold is 0.2 m/s and lift-off threshold 0.05 m/s. [8] This means an estimator can already know that lift has ended while the audio layer continues to present an obsolete lift cue.

That should be explicitly forbidden in ours:

> **STOP is an asynchronous audio event. It may interrupt the current beep.**

The tone can be faded over a few milliseconds to prevent clicks, but it should never have to finish a 200–500 ms scheduled beep before responding to an exit decision.

There is striking anecdotal support for this priority. In a 2015 German paragliding-forum comparison, a pilot reported that an XC Tracer stopped sounding immediately when leaving lift, while a BlueFly lifted roughly one metre on the ground reportedly continued beeping for about **1.5 seconds**. A later pilot reported that a sensitively configured BlueFly could approach XC Tracer on initial response but still "after-beep" for roughly **0.5 s**. These are uncontrolled pilot observations, not instrumented benchmarks, but they align closely with the audio/filter behaviour documented in BlueFly's own manual. [9]

The main architectural conclusion is therefore:

Design question	Research conclusion
More smoothing to stop false beeps?	**Wrong primary strategy.** It buys noise reduction with latency.
Barometer-only Kalman?	Good fallback and essential baseline, but fundamentally limited because velocity must be inferred from noisy altitude history.
Complementary baro + acceleration?	Excellent transparent baseline; academically well-supported.
EKF/UKF for the entire vertical channel?	Usually unnecessary. Use non-linear estimation where non-linearity exists—primarily attitude.
Linear KF after Earth-frame acceleration exists?	**Best primary estimator.**
State `[h,v,a]`?	Useful baro-only model, but acceleration estimated from barometric history remains noisy.

Design question	Research conclusion
State `[h,v,b_a]` with IMU acceleration as input?	**Recommended.** Direct high-frequency motion input plus barometric drift/bias correction.
Fast + slow filters?	**Yes, in parallel.** Fast drives audio; slow supplies confidence/ fallback.
Cascade several smoothing filters?	Avoid. Cascading low-pass stages compounds delay.
Dual barometers?	Useful for electronic-noise reduction, diagnostics and robustness, but not the strongest latency lever.
IMU fusion?	**Much more likely to reduce latency**, because it adds an independent high-frequency observation of motion.
Internal BMP58x IIR?	Bypass initially.
Exit audio?	Interruptible, fast-release state machine.
ESP32-S3?	More than capable; timing discipline matters more than MCU compute.

The biggest finding is therefore slightly different from “find the perfect Kalman tuning”:

> **The best instant variometer is not one aggressive filter. It is a carefully timed system in which IMU prediction gives immediate motion information, the barometer continuously corrects drift, a slower estimator measures confidence rather than slowing the fast estimate, and the audio layer is allowed to stop immediately.**

Estimation theory and the real latency problem

Why a pressure-only variometer inevitably faces a noise–latency trade

A barometer directly measures pressure, from which altitude can be inferred. It does **not** directly measure vertical velocity. A pressure-only vario must therefore infer velocity from changes in altitude over time.

Naively,

$$v_z(t) \approx \frac{h(t) - h(t - \Delta t)}{\Delta t}$$

but differentiation strongly amplifies high-frequency measurement noise. The obvious fix is low-pass filtering. That suppresses noise but introduces temporal memory—and therefore latency.

This is not merely theoretical. Bosch's BMP585 implements

$$y_k = \frac{C\, y_{k-1} + x_k}{C + 1}$$

for its internal IIR. Bosch explicitly states that low-pass filtering reduces bandwidth and publishes step-response curves showing progressively slower response as C increases. [7]

From that equation, for a unit step and initially zero output,

$$y_n = 1 - \left(\frac{C}{C+1}\right)^n$$

so the delay can be quantified directly.

For example, at BMP585's ×16 recommended setting of approximately 87 Hz, the sensor-internal IIR alone would have approximately:

IIR coefficient	Samples to 90% of step	Approx. time at 87 Hz	Interpretation
bypass	~0	~0 ms additional IIR settling	Recommended starting point
1	3.3	38 ms	Mild but measurable
3	8.0	92 ms	Already material
7	17.2	198 ms	Too large for our target
15	35.7	410 ms	Conventional “smooth vario” territory
31	72.5	834 ms	Clearly inappropriate for instant audio

These values are calculated from Bosch's published IIR equation and nominal 87 Hz ODR; they represent **only the sensor IIR step response**, before the Kalman filter, threshold logic, USB path or audio scheduler. Bosch's published curves show the same qualitative behaviour. [10]

That explains your original intuition very well:

Every time we solve pressure noise by adding more temporal averaging, we spend latency.

Linear Kalman filtering

The original Kalman formulation estimates the internal state of a linear dynamic system recursively from a process model and noisy observations, while propagating an estimate of the uncertainty covariance. [11]

A minimal pressure-only vertical model is

$$\mathbf{x} = \begin{bmatrix} h \\ v \end{bmatrix}$$

with

$$\mathbf{x}_{k+1} = \begin{bmatrix} 1 & \Delta t \\ 0 & 1 \end{bmatrix} \mathbf{x}_k + \mathbf{w}_k$$

and barometric altitude measurement

$$z_k = \begin{bmatrix} 1 & 0 \end{bmatrix} \mathbf{x}_k + n_k.$$

This is considerably better than simply differentiating filtered altitude because the filter explicitly maintains estimates of both altitude and velocity together with their covariance.

For a continuous white-acceleration process with spectral density q_a, a common process-noise model for this state is

$$Q = q_a \begin{bmatrix} \Delta t^3/3 & \Delta t^2/2 \\ \Delta t^2/2 & \Delta t \end{bmatrix}.$$

The pressure-derived altitude measurement covariance is

$$R = \sigma_h^2.$$

The practical behaviour is dominated by the ratio between model/process uncertainty Q and measurement uncertainty R:

larger effective R → trust pressure less → smoother but slower response to pressure changes;

larger effective Q → admit faster real dynamics → more responsive, but potentially noisier.

This is exactly why there is no universal “best Kalman parameter”. The correct values depend on the actual barometer noise, pressure-port environment, IMU quality, mounting and the motion spectrum we want the instrument to reproduce. BlueFly exposes a `positionNoise` setting for this reason; its current manual describes higher values as reducing sensitivity by smoothing noisy pressure measurements. [12]

Adding acceleration as a state

Another pressure-only model is

$$\mathbf{x} = \begin{bmatrix} h \\ v \\ a \end{bmatrix},$$

with constant-acceleration transition

$$F = \begin{bmatrix} 1 & \Delta t & \frac{1}{2}\Delta t^2 \\ 0 & 1 & \Delta t \\ 0 & 0 & 1 \end{bmatrix}.$$

Altitude measurements over successive samples theoretically make h, v, a observable for non-zero Δt, but in practice the acceleration component is being extracted from the **second derivative of noisy altitude**. On the short time horizon where an instant vario needs acceleration information most, it is poorly conditioned.

It helps prediction, but it does not magically create a high-quality motion observation.

That is why an actual accelerometer changes the problem much more fundamentally than simply adding another state.

The recommended vertical state

Once an IMU has supplied gravity-compensated Earth-frame vertical acceleration, I recommend

$$\mathbf{x} = \begin{bmatrix} h \\ v \\ b_a \end{bmatrix},$$

where b_a is the residual vertical accelerometer bias.

Using measured vertical acceleration a_m as an input,

$$\dot{h} = v$$

$$\dot{v} = a_m - b_a + w_a$$

$$\dot{b}_a = w_b.$$

A convenient discrete prediction is approximately

$$h_{k+1} = h_k + v_k \Delta t + \frac{1}{2}(a_m - b_a)\Delta t^2$$

$$v_{k+1} = v_k + (a_m - b_a)\Delta t$$

$$b_{a,k+1} = b_{a,k}.$$

Barometric altitude then supplies the correction

$$z_h = h + n_h.$$

This architecture is very close in spirit to the open-source `Kalmanfilter_altimeter_vario` implementation, whose state includes altitude, vertical velocity and residual accelerometer bias. Its related ESP32 projects were created specifically to fuse high-rate IMU and lower-rate barometric information for responsive variometer output. [13]

The important point is that the accelerometer now supplies **high-frequency prediction**, while the barometer anchors the low-frequency solution and prevents acceleration bias from integrating indefinitely.

That changes the latency problem from:

> “How much pressure smoothing can we tolerate?”

to:

> “How rapidly can we trust the inertial prediction before barometric confirmation arrives?”

That is a far better problem to have.

Observability and accelerometer bias

The bias state deserves special attention.

Even a tiny residual gravity/orientation error matters. A persistent vertical-acceleration error of only 0.01 m/s² produces a velocity error of roughly 0.1 m/s after ten seconds if left uncompensated.

Barometric altitude constrains this bias because incorrect bias eventually produces inconsistent predicted velocity/altitude. That makes residual vertical bias estimable over time, but **practical observability can be weak over short intervals** because accelerometer bias, real sustained acceleration and barometric disturbance can mimic one another.

This is why I would not try to make the IMU carry low-frequency altitude indefinitely. The architecture should deliberately allow:

- IMU dominance at short timescales;
- barometer dominance at long timescales;
- a slowly varying acceleration-bias state connecting the two.

The open-source har-in-air implementation takes exactly this bias-aware approach. [13]

Complementary filters

A complementary filter expresses this separation directly.

At a conceptual level:

$$\hat{h}(s) = LPF\{h_{\text{baro}}\} + HPF\left\{\frac{a_z}{s^2}\right\}.$$

Low-frequency altitude comes from pressure because an accelerometer drifts when integrated. High-frequency motion comes from acceleration because pressure-derived velocity is noisy and requires temporal history.

Sabatini and Genovese use a cascaded scheme in which a quaternion EKF first determines orientation; the resulting gravity-compensated vertical acceleration and pressure altitude then feed a complementary filter that estimates vertical height and velocity. They additionally investigated whitening short-term correlated pressure disturbances rather than treating all barometric error as independent white noise. [14]

Lee subsequently proposed another two-stage Kalman/complementary architecture: a first filter estimates vertical acceleration from six-axis IMU measurements and orientation, and a second complementary stage combines that acceleration with barometric information. [15]

A complementary filter therefore makes an excellent **reference implementation** for us because its crossover is easier to interpret than Kalman Q/R tuning.

If the sophisticated KF cannot clearly beat a properly tuned complementary filter on the same raw dataset, we should distrust the sophisticated filter rather than assume complexity equals performance.

EKF versus UKF versus linear KF

An EKF is useful where the state/measurement relationship is genuinely nonlinear—most obviously orientation. Rotation of body-frame accelerometer measurements into a navigation frame using a quaternion and subtracting gravity is nonlinear. Sabatini therefore used a quaternion EKF for this stage. [2]

The Unscented Kalman Filter avoids the EKF's explicit first-order Jacobian linearisation by propagating a set of sigma points through nonlinear transformations. Julier and Uhlmann describe its advantages for nonlinear estimation where first-order EKF approximations become problematic. [16]

But **our vertical dynamics after attitude correction are almost entirely linear**.

That leads to this recommendation:

Filter	Strength	Weakness	Role in our vario
Moving average / IIR	Very simple	Direct group delay, poor dynamics	Diagnostic baseline only
Baro-only 2-state KF `[h,v]`	Robust, easy, interpretable	Velocity must come from pressure history	Essential fallback/ baseline
Baro-only 3-state KF `[h,v,a]`	Better dynamic prediction	`a` inferred from noisy baro	Experimental baseline
Complementary baro+IMU	Transparent frequency split; fast	Fixed crossover unless made adaptive	**Excellent baseline**
Linear fused KF `[h,v,b_a]`	Fast IMU prediction + baro stability + bias estimation	Needs good Earth-Z acceleration	**Recommended main estimator**
EKF for whole vertical problem	Can directly represent nonlinear states	More tuning/debug complexity	Unnecessary unless tightly coupled
Quaternion EKF/attitude filter → linear vertical KF	Places nonlinearity where it actually exists	Two stages to validate	**Recommended structure**
UKF	Handles stronger nonlinearities without Jacobians	More compute/ state/tuning complexity	Research option, not first choice

The ESP32-S3 has enough compute for a UKF, but **compute headroom is not an argument for using one**. We should choose the estimator whose failure modes and tuning we can most clearly measure.

Cascaded and stacked estimator architectures

There are two ideas that are often both called “multiple filters”, but they serve very different purposes.

Cascaded filters

A good cascade decomposes distinct physical estimation problems:

orientation estimator → gravity cancellation → vertical-motion estimator.

That is the structure used by Sabatini and Genovese and by the two-step Lee architecture. [17]

A bad cascade looks like:

barometer IIR → moving average → Kalman → app smoothing → audio averaging.

Each stage may appear reasonable when examined independently, but their phase delays accumulate.

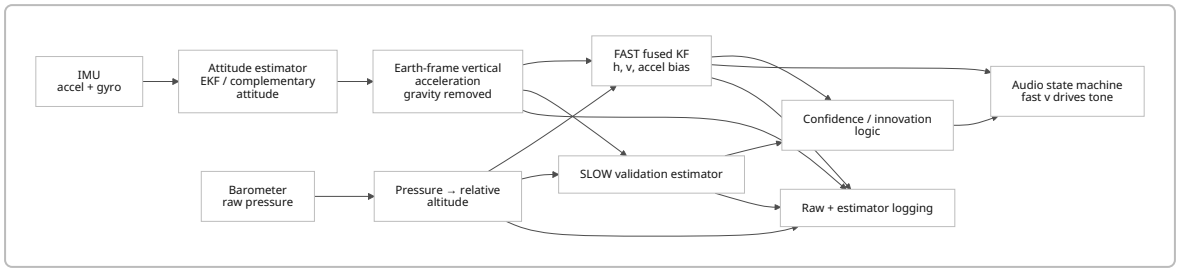
A very important design rule for this project is therefore:

> **Cascade estimators when each stage estimates a different hidden quantity. Do not casually cascade multiple filters whose only job is smoothing the same pressure signal.**

Targeted pressure whitening is different from generic smoothing. Sabatini's work is interesting precisely because the pressure-conditioning stage attempted to model/remove short-term correlated barometric disturbance, rather than blindly applying another low-pass. [2]

Parallel fast and slow estimators

For our vario I see considerable value in running **two vertical estimators concurrently**.



The critical detail is the last part:

FAST → audio directly.

The slow bank should not be averaged 50/50 with the fast bank, because that defeats the reason for having a fast bank.

Instead, the slow bank asks questions such as:

- Is the persistent trend compatible with the fast estimate?
- Has the fast filter covariance become abnormally large?
- Is the barometer innovation behaving plausibly?
- Did a transient acceleration spike occur without any subsequent pressure evidence?

- Has the IMU become untrustworthy?
- Is there a pressure-port disturbance affecting only one sensor?

This lets us obtain **fast response without blindly trusting every high-frequency event**.

Innovation-based confidence

For a Kalman altitude correction,

$$\nu = z - H\hat{x}^-$$

is the innovation and

$$S = HP^-H^T + R$$

is its expected covariance.

For the scalar barometric measurement,

$$NIS = \frac{\nu^2}{S}$$

provides a useful normalised measure of how surprising the pressure observation is.

This gives us a better way to think about confidence than “smooth more”.

A confidence score can incorporate:

$$C = f(P_{vv}, NIS, \text{sample age}, \text{IMU health}, \text{attitude quality}, \text{sensor agreement})$$

where P_{vv} is estimated vertical-velocity variance.

We do not need to turn this into an elaborate machine-learning system. The primary filter can remain conventional and deterministic.

Fast/slow bank trade-offs

Architecture	Benefit	Main danger	Recommendation
Single aggressive KF	Minimum complexity	False cues when input becomes abnormal	Baseline
Single conservative KF	Stable output	Loses the whole instant-vario advantage	Fallback only
Fast KF + averaged slow KF	Smoother	Reintroduces latency	Avoid
Fast KF + slow confidence observer	Fast output, extra robustness	More tuning/logic	**Recommended**

Architecture	Benefit	Main danger	Recommendation
Multiple differently tuned KF banks + selector	Powerful for regime changes	State switching can itself create artefacts	Research after baseline
Adaptive R based on baro innovation	Can reject pressure disturbances	May incorrectly suppress a real thermal	Use cautiously
Dual barometers feeding one KF	Reduces independent sensor noise	Common-mode airflow remains	Useful secondary improvement

A related multi-filter concept appears outside paragliding in multisensor vertical-state estimation, where multiple Kalman instances over different histories can be evaluated against integrity/accuracy criteria. Such work supports the general feasibility of filter-bank architectures, but it should be regarded as architectural inspiration rather than evidence that a particular bank configuration is optimal for a paraglider. [18]

Why I would not start with a UKF bank

It would be easy on an ESP32-S3 to run several EKFs/UKFs simultaneously.

That does not make it a good first experiment.

Our biggest unknowns are presently:

actual pressure disturbances, accelerometer bias, mounting orientation, airflow artefacts, useful audio thresholds and human-perceived release timing.

A transparent linear KF gives us clear innovations, covariance and tuning parameters. That makes problems diagnosable.

Only move to a UKF if replay data demonstrates that a nonlinear model produces measurable performance that the simpler architecture cannot.

Sensors, sampling, timing and practical tuning

MS5611: useful development sensor, not necessarily the final winner

The MS5611 remains extremely useful because we already have three of them.

TE designed it for altimeter/variometer applications and provides selectable oversampling with this speed/resolution trade-off. [5]

MS5611 OSR	Typical conversion time	Max specified conversion time	RMS pressure resolution
256	0.54 ms	0.60 ms	6.5 Pa

MS5611 OSR	Typical conversion time	Max specified conversion time	RMS pressure resolution
512	1.06 ms	1.17 ms	4.2 Pa
1024	2.08 ms	2.28 ms	2.7 Pa
2048	4.13 ms	4.54 ms	1.8 Pa
4096	8.22 ms	9.04 ms	1.2 Pa

TE's datasheet also makes clear that temperature and pressure are separate D2/D1 conversions and temperature is required for pressure compensation. [19]

For our first dataset I would therefore test at least:

MS5611 OSR4096 at approximately 50 Hz as the high-resolution conventional baseline;

OSR2048 at a higher pressure update rate to see whether lower conversion latency produces a better final *velocity* estimate despite noisier individual pressure samples.

Do not assume that the maximum-OSR setting is automatically superior for an instant vario.

BMP581/BMP585: the more interesting final sensor

The BMP581 and BMP585 make the trade-off unusually explicit. Bosch's recommended settings include: [6]

Pressure OSR	Typical pressure noise	Typical continuous ODR	My view
×1	0.78 Pa	498 Hz	Very fast, unnecessarily noisy
×2	0.58 Pa	374 Hz	Interesting experiment
×4	0.41 Pa	255 Hz	Strong fast mode candidate
×8	**0.30 Pa**	**155 Hz**	**Excellent instant-vario candidate**
×16	**0.21 Pa**	**87 Hz**	**Excellent conservative candidate**
×32	0.15 Pa	46 Hz	Starting to sacrifice too much bandwidth
×64	0.11 Pa	24 Hz	Too slow for our objective
×128	0.08 Pa	12 Hz	Wrong optimisation target for instant audio

Bosch makes another extremely important observation: by the standard-resolution range, **real fluctuations of ambient air pressure may exceed intrinsic sensor noise**. Higher ODR or lower electronic noise cannot remove those low-frequency environmental disturbances. [20]

That point has direct architectural consequences.

It means moving from 0.21 Pa to 0.15 Pa sensor noise may contribute less than improving:

- pressure-port geometry;
- airflow shielding;
- thermal isolation;
- mounting location;
- IMU-assisted motion discrimination.

That is also why dual-barometer averaging has finite value.

Dual barometer versus IMU fusion

If two pressure sensors contain equal, independent random electronic noise with variance σ^2, averaging them gives

$$\sigma_{\text{avg}} = \frac{\sigma}{\sqrt{2}}.$$

Thus two approximately 0.21 Pa RMS sensors could theoretically produce about 0.15 Pa RMS independent electronic noise after averaging.

But there is a crucial limitation:

$$p_1 = p_{\text{true}} + p_{\text{airflow}} + p_{\text{thermal}} + n_1$$

$$p_2 = p_{\text{true}} + p_{\text{airflow}} + p_{\text{thermal}} + n_2.$$

Averaging cancels some n_1, n_2.

It does **not** cancel common p_{airflow} or p_{thermal}.

Bosch's statement that atmospheric fluctuations can already dominate intrinsic sensor noise supports exactly this concern. [6]

By contrast, an accelerometer gives us a genuinely different observation modality.

Therefore my expected ranking for reducing onset/release latency is:

IMU fusion > better single barometer configuration > dual-barometer averaging.

Dual barometers remain attractive for fault detection, comparative port testing and robustness—but I would not make dual pressure sensors our main latency strategy.

MPU6050

The MPU6050 is old, but it is perfectly adequate for determining whether baro/IMU fusion provides the response advantage we seek.

TDK/InvenSense's MPU-6000/6050 register documentation provides programmable sample-rate division and digital low-pass filtering; the accelerometer data path is capable of rates well beyond the few hundred hertz we require. [21]

For the first ESP32-S3 prototype I recommend:

IMU acquisition: 200 Hz

rather than immediately pursuing 500 or 1000 Hz.

A 5 ms IMU interval is already much shorter than the likely barometric, detection and audio timescales. Once the pipeline works, testing 500 Hz is easy.

The open-source har-in-air variometer/logging projects run their IMU at 500 Hz and barometer at 50 Hz, demonstrating that this sort of rate separation is entirely practical. [13]

Timestamping matters more than a perfectly periodic task

Do not design the estimator as though

```
dt = 0.005000000
```

simply because the task was nominally configured for 200 Hz.

Instead:

timestamp the physical sample as closely as possible and use measured Δt.

For an IMU with data-ready:

> sensor DRDY → timestamp immediately → queue sample → process later.

For an MS5611:

> conversion command timestamp → conversion → ADC read timestamp.

A reasonable effective measurement timestamp can be associated with the conversion interval rather than pretending the ADC value was produced at the time the firmware happened to read it.

For BMP58x:

> use data-ready/FIFO signalling where practical and timestamp acquisition consistently.

ESP-IDF's GPTimer on the ESP32-S3 is a dedicated hardware timer intended for high-resolution timestamps and periodic alarms, so there is no need to derive measurement time from FreeRTOS ticks. [22]

Asynchronous Kalman updates

The barometer and IMU **should not be artificially forced to the same sample rate**.

At every IMU measurement:

$$\text{predict}(\Delta t, a_z)$$

At every barometer measurement:

$$\text{correct}(h_{\text{baro}})$$

This is the natural Kalman architecture.

For example:

```
IMU  t=0       predict
IMU  t=5 ms    predict
IMU  t=10 ms   predict
BARO t=12 ms   pressure correction
IMU  t=15 ms   predict
IMU  t=20 ms   predict
BARO t=23 ms   pressure correction
...
```

There is no reason to hold IMU samples waiting for the next pressure measurement.

That would throw away the very high-frequency information we added the IMU to obtain.

Starting noise models from measurement rather than guesswork

Do not begin by searching online for someone's "magic" Q and R.

Measure them.

For barometric altitude:

$$R_h \approx \text{Var}(h_{\text{baro}} - \bar{h})$$

during a stationary controlled test.

For acceleration:

$$\sigma_{a_z}^2 = \text{Var}(a_z)$$

after attitude rotation and gravity subtraction.

Then run much longer static/thermal sessions to characterise bias drift separately from short-term measurement noise.

Datasheet figures give a sanity check, not the final covariance. Bosch itself warns that environmental air-pressure fluctuations can exceed intrinsic device noise. [23]

The specific BMP58x configuration I would start with

For a future BMP581/BMP585 prototype:

Parameter	Initial configuration
Pressure OSR	**×8**
Temperature OSR	×1
Operating mode	Continuous
Nominal pressure ODR	Up to ~155 Hz according to Bosch recommended combination
Firmware target rate	100–150 Hz
Pressure IIR	**Bypass**
FIFO	Optional, primarily for robust acquisition
IMU	200–500 Hz
Fusion update	Every IMU sample; baro corrections asynchronously

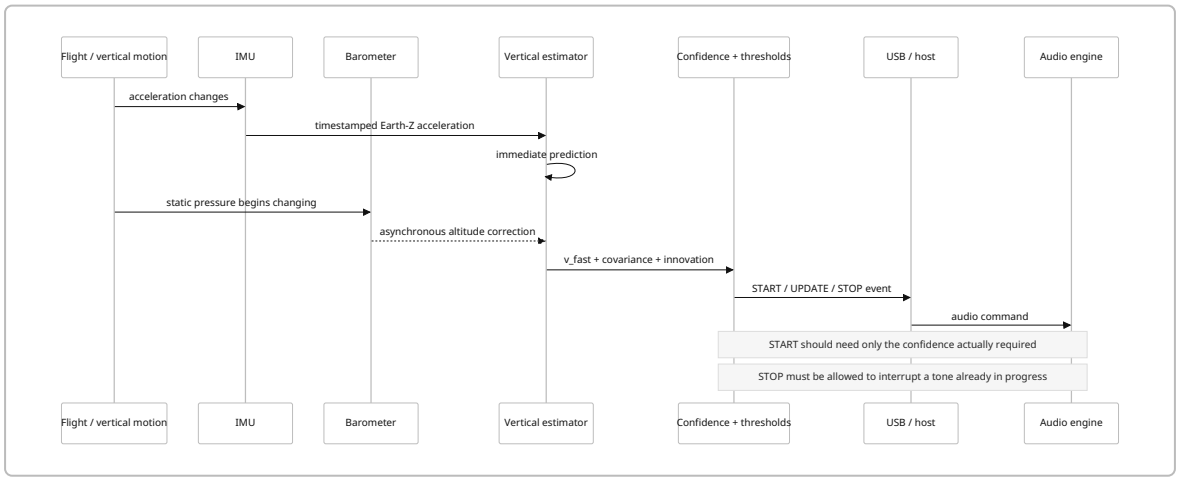
The second configuration to test should be **×16 / approximately 87 Hz**. Bosch specifies 0.21 Pa RMS there versus 0.30 Pa at ×8, so comparing those two modes will tell us far more than blindly choosing the lowest-noise ×128 mode. [24]

Audio state machine and end-to-end responsiveness

This is where I think we can make a product materially better than many conventional implementations.

Estimator latency and audio latency are different

The complete path is:



The complete perceptual latency is approximately

$$T_{\text{audio}} = T_{\text{sensor}} + T_{\text{estimator}} + T_{\text{decision}} + T_{\text{transport}} + T_{\text{audio-buffer}}.$$

If we reduce estimator delay by 100 ms but introduce 200 ms in the tone scheduler, we have achieved nothing useful.

BlueFly gives us an unusually clear example

The July 2026 BlueFlyVario manual states:

- pressure is sensed 50 times per second;
- filtering introduces a slight delay between physical movement and sensed lift;
- filtered lift is recalculated every 20 ms;
- the default lift-on threshold is 0.2 m/s;
- the default lift-off threshold is 0.05 m/s;
- after a lift beep begins, it runs for its cadence-defined duration;
- a silence period follows;
- only then can the beeping sequence end. [25]

The manual's own beep-duration plot also shows that weak-lift beeps can themselves last a substantial fraction of a second. [26]

This is exactly the failure mode you were describing:

> **The vertical-speed estimate may already have fallen, but the instrument is still physically playing an old beep.**

That is unacceptable for our “instant” definition.

Recommended lift state machine

I would separate **estimator state** from **audio state**.

Conceptually:

```
SILENT
  ↓ credible rise
LIFT_ARMING
  ↓ confidence achieved
LIFT
  ↓ v < off threshold
RELEASE_CONFIRM
  ↓ confirmed
SILENT
```

But strong lift should be able to skip or drastically shorten `LIFT_ARMING`.

Likewise, `RELEASE_CONFIRM` must be short.

A sensible first bench configuration—not a flight-proven final tuning—is:

Setting	Starting point	Reason
Lift-on	+0.20 m/s	Conveniently matches BlueFly/XC-Tracer-type conventional starting threshold
Lift-off	+0.05 m/s	Hysteresis without requiring descent
Strong-lift immediate trigger	+0.5 m/s	No reason to wait when evidence is strong
Weak-lift confirmation	40–80 ms	Suppress one/two-sample events without hundreds of ms smoothing
Release confirmation	20–50 ms	Prioritise rapid exit information
Audio stop ramp	~5–15 ms	Avoid acoustic click, not perceptible “after-beep”
Stale-data timeout	Explicit	Sensor/USB failure should silence or enter fail state, never hold old lift

The threshold pair is consistent with the current BlueFly defaults of +0.20/+0.05 m/s, but the dwell/ramp values above are our proposed test starting points rather than copied commercial settings. [27]

Interruptible tones

This is the key implementation difference.

Suppose a 300 ms lift beep starts at $t = 0$.

At $t = 90$ ms the estimator concludes lift has ended.

A conventional cadence scheduler may do:

```
0 ms                 300 ms
|----------------------|
      beep continues
```

Our scheduler should do:

```
0 ms       90 ms  100 ms
|-----------|------|
   beep      fade
             STOP
```

Audio cadence should describe the **current** estimator state, not become a queue of historical decisions.

Confidence should affect onset more than release

A particularly dangerous design is:

Fast to latch into lift, slow to unlatch because the “stable” filter has not yet caught up.

That produces exactly the persistent-beeping behaviour we dislike.

Instead:

Onset: fast estimator + appropriate confidence.

Release: fast estimator crossing the release condition, with only short deglitching.

Slow bank: may veto obviously impossible onset spikes, but should never hold a lift beep alive merely because it is lagging behind.

This is an important difference between using a slow filter as a **confidence observer** and using it as an audio smoother.

USB/phone audio must be benchmarked independently

The ESP32-S3 supports USB Full-Speed device operation and TinyUSB classes including CDC, so the dongle can transmit timestamped estimates/events to the host without Bluetooth being part of the critical path. [28]

But the phone's audio system creates a second experiment.

We should therefore log:

$$t_{\text{physical reference}}$$

$$t_{\text{sensor}}$$

$$t_{\text{estimate crossing}}$$

$$t_{\text{USB event transmitted}}$$

$$t_{\text{audio actually audible}}.$$

The last quantity should be measured with a microphone or electrical loopback, not inferred from the app calling an audio API.

That will allow us to say, for example:

sensor/estimator release = 72 ms
USB/app = 18 ms
audio engine = 61 ms
total = 151 ms

rather than blaming “the Kalman filter” for an end-to-end delay generated elsewhere.

Evidence from research papers, open source and pilots

What has actually been measured

A major finding of this research is that **published quantitative onset/release latency data are surprisingly scarce**.

I did not find a controlled primary publication saying, for example:

> XC Tracer onset = 73 ms, release = 81 ms.

Nor did I find an equivalent controlled BlueFly measurement.

What we do have is a combination of estimator-accuracy research, manufacturer architecture claims, open-source high-rate recordings and pilot observations.

Source	What was demonstrated	Quantitative evidence	Latency evidence
Sabatini & Genovese, 2014	EKF attitude + complementary baro/ acceleration vertical estimator	Velocity RMSE 0.04–0.24 m/s; height RMSE 5–68 cm	No paraglider onset/release number [2]
Lee, 2016	Two-stage KF/ complementary IMU–baro estimator	Mean height error 19.7 cm vs 43.4 cm raw barometer	No instant-vario release benchmark [3]
Brzozowski et al., 2018	Cascaded Kalman/ complementary fusion for hang-glider/ paraglider instrument	Laboratory/flight prototype research	Accessible abstract does not provide a clean ms onset/release metric [4]
har-in-air ESP32 vario	Real-time baro+IMU KF; offline replay	500 Hz IMU, 50 Hz baro; downloadable recordings/ notebooks	Developer describes response as “zero-lag”, not independent laboratory latency measurement [29]
BlueFlyVario	Pressure KF + configurable thresholds/ cadence	50 Hz pressure/filter cycle	Manufacturer explicitly acknowledges filter delay and cadence-constrained stop behaviour [30]
XC Tracer	IMU + barometer + GPS sensor fusion	Commercial architecture	Manufacturer calls indication lag/delay-free but publishes no controlled ms figure on the pages found [31]
Pilot ground comparison	XC Tracer vs BlueFly	BlueFly reportedly continued ~1.5 s after a 1 m lift	Anecdotal, uncontrolled [32]

Source	What was demonstrated	Quantitative evidence	Latency evidence
Later pilot report	Tuned BlueFly vs XC Tracer	BlueFly reportedly ~0.5 s after-beep while onset nearly as fast	Anecdotal, configuration-dependent [33]

This means **our own instrumented onset/release dataset could be unusually valuable**.

XC Tracer

XC Tracer's current product information explicitly lists accelerometer, compass, gyro, barometer and GPS and describes its climb/descent indication as lag-free. Its technical explanation says conventional pressure-based varios require filtering and therefore exhibit delay, while it combines multiple sensor streams mathematically to eliminate this delay. [31]

Its configuration also permits damping; the available Mini V documentation states that adding damping deliberately makes the variometer react more slowly. [34]

The architecture is therefore strongly consistent with what the academic literature suggests.

What XC Tracer does **not** publicly provide is enough information to clone its exact estimator: no complete production state vector, covariance model, gain schedule or measured onset/release timing is published in the material found.

So our objective should not be “copy XC Tracer's Kalman filter”.

It should be:

> **measure whether our estimator/audio pipeline achieves the same qualitative property pilots praise: fast onset without nervous audio and fast cessation when lift ends.**

The most useful XC Tracer forum observation

A 2015 pilot in the German Gleitschirm/Drachen forum wrote after roughly six hours of XC Tracer flying that it responded to small altitude changes very quickly and, importantly, stopped the tone immediately after leaving lift rather than continuing to “after-beep”. In a ground comparison the same discussion reports that BlueFly onset was only fractionally slower, but after a roughly one-metre lift BlueFly reportedly kept sounding for about 1.5 seconds whereas XC Tracer did not. [32]

Another forum participant several years later wrote that a sensitively tuned BlueFly could respond almost as quickly as XC Tracer but continued to beep for approximately 0.5 seconds after the movement. [33]

These reports are **not controlled data**. Different firmware, tuning, lift amplitudes and beep cadence make direct comparison impossible.

But their most interesting feature is that pilots distinguish:

initial response

from

after-beeping.

That validates our decision to treat them as two separate product metrics.

BlueFly's current documentation confirms the mechanism

The pilot anecdotes would be weak evidence by themselves.

The much stronger evidence is that BlueFly's own 2026 manual explicitly describes:

1. filtering-related movement-to-lift delay;
2. separate lift-on/lift-off hysteresis;
3. cadence-length beeps;
4. the fact that beeping can only end after the active cadence/silence sequence reaches the relevant decision point. [35]

That makes BlueFly an extremely useful engineering reference, because the developer documents the trade rather than hiding it.

Open-source datasets we should actually use

The highest-value repository I found is:

ESP32 IMU BARO GPS VARIO

`https://github.com/har-in-air/ESP32_IMU_BARO_GPS_VARIO`

It records IMU data at 500 Hz, barometric data at 50 Hz and GPS at 10 Hz and includes offline Jupyter analysis comparing Kalman variants on captured sensor data. [29]

Its associated logger:

ESP32 IMU BARO GPS LOGGER

`https://github.com/har-in-air/ESP32_IMU_BARO_GPS_LOGGER`

contains an example binary sensor log plus extraction/processing code intended specifically for offline estimator development and comparison between barometer-only and acceleration-fused Kalman approaches. [36]

The underlying filter:

Kalmanfilter_altimeter_vario

`https://github.com/har-in-air/Kalmanfilter_altimeter_vario`

uses altitude, vertical velocity and accelerometer-bias states and is therefore particularly close to the architecture I recommend.

Another directly useful implementation is:

AltitudeEstimation

`https://github.com/juangallostra/AltitudeEstimation`

which implements the Lee two-stage Kalman/complementary IMU/barometer architecture in hardware-independent C++. [37]

These repositories are much more valuable to us than random “Arduino vario” code because they include either estimator documentation, offline analysis or real raw recordings.

Primary research and technical links

The papers I would preserve in the project research directory are:

Sabatini & Genovese — A Sensor Fusion Method for Tracking Vertical Velocity and Height Based on Inertial and Barometric Altimeter Measurements

`https://www.mdpi.com/1424-8220/14/8/13324`

[2]

Lee — A Two-step Kalman/Complementary Filter for Estimation of Vertical Position Using an IMU-Barometer System

`https://www.kci.go.kr/kciportal/ci/sereArticleSearch/ciSereArtiView.kci?sereArticleSearchBean.artiId=ART002111843`

[3]

Brzozowski et al. — Measurement data fusion with cascaded Kalman and complementary filter in the flight parameter indicator for hang-glider and paraglider

`https://www.sciencedirect.com/science/article/pii/S0263224118300976`

[4]

Julier & Uhlmann — Unscented Filtering and Nonlinear Estimation

`https://ieeexplore.ieee.org/document/1271397`

[16]

Kalman — A New Approach to Linear Filtering and Prediction Problems

`https://doi.org/10.1115/1.3662552`

11

Manufacturer references:

Bosch BMP581 datasheet

`https://www.bosch-sensortec.com/media/boschsensortec/downloads/datasheets/bst-bmp581-ds004.pdf`

38

Bosch BMP585 datasheet

`https://www.bosch-sensortec.com/media/boschsensortec/downloads/datasheets/bst-bmp585-ds003.pdf`

39

TE MS5611 product/datasheet

`https://www.te.com/en/product-MS561101BA03-50.html`

40

TDK/InvenSense MPU6050 technical resources

`https://www.invensense.tdk.com/en-us/search-result?query=6050`

41

ESP-IDF GPTimer

`https://docs.espressif.com/projects/esp-idf/en/stable/esp32s3/api-reference/peripherals/gptimer.html`

22

ESP32-S3 USB device stack

`https://docs.espressif.com/projects/esp-usb/en/latest/esp32s3/usb_device.html`

28

Commercial/developer references:

BlueFlyVario Hardware Settings Manual V2.0

`https://www.blueflyvario.com/files/BFV_HardwareSettings_Manual_v2.0.pdf`

42

XC Tracer Mini V official product information

`https://www.xctracer.com/en/xctracerminiiiigps`

43

Forum evidence, to be filed separately as anecdotal rather than engineering specification:

`https://www.gleitschirmdrachenforum.de/forum/gleitschirm-und-drachen-forum/technik/32244-xc-tracer`

32

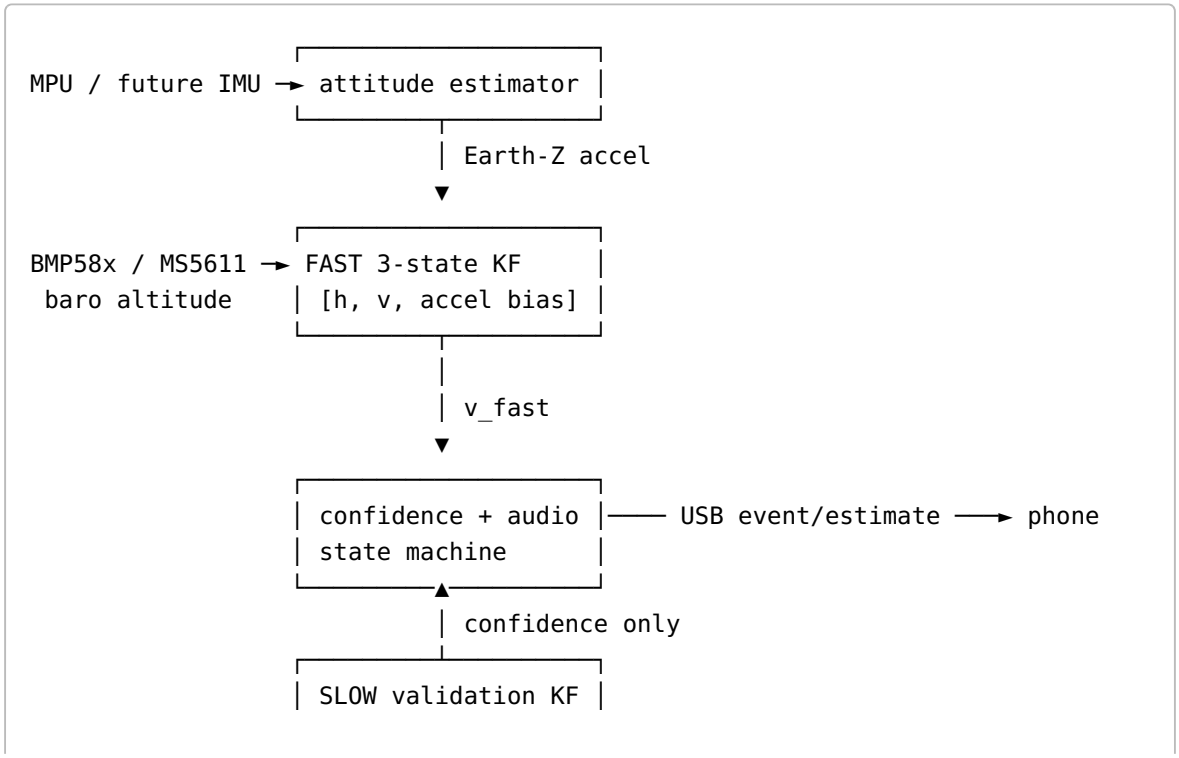
`https://www.gleitschirmdrachenforum.de/forum/gleitschirm-und-drachen-forum/technik/35933-minivarios-%C3%BCbersicht-praktische-erfahrungen`

33

Recommended USB-C estimator and validation programme

Recommended shipping architecture

For our ESP32-S3 USB-C instant vario I would now commit the *development architecture* as follows:



```
                    | / baro-only KF      |
                    └─────────────────────┘
```

Primary estimator: IMU-fused linear KF `[altitude, vertical velocity, acceleration bias]`.

Attitude stage: initially a conventional quaternion attitude solution; EKF is academically well-founded, but a good lightweight complementary/Mahony-style attitude estimator could also be benchmarked because our purpose is accurate gravity removal rather than globally perfect navigation.

Validation estimator: independent two-state baro-only `[h,v]` KF, deliberately more conservative.

Complementary baro/IMU filter: implement as an experimental baseline because it is simple, interpretable and well-supported by the literature. [17]

UKF: do not implement until the above have been replayed against real data and a specific deficiency has been demonstrated.

Development sampling configuration

With hardware already on the bench:

Component	Development rate	Configuration
MPU6050	**200 Hz**	accel + gyro, data-ready if accessible
MS5611	**50 Hz initially**	OSR4096 baseline
MS5611 experiment	80–100+ Hz where practical	OSR2048/lower OSR comparison
Fast KF prediction	**200 Hz**	every IMU sample
Baro correction	**50 Hz** initially	asynchronous
Slow validation KF	50 Hz	barometric measurement cadence
USB logging	buffered	never block acquisition
Raw log	every measurement	never log only filtered result

For the BMP581/BMP585 stage:

Component	Recommended first experiment
Barometer	BMP581 or BMP585
OSR	**×8 pressure / ×1 temperature**
Approx. sensor capability	0.30 Pa RMS / 155 Hz continuous
Firmware target	~100–150 Hz
Internal IIR	**bypass**

Component	Recommended first experiment
Comparison	×16 / 0.21 Pa / ~87 Hz
IMU	200 Hz first, then 500 Hz A/B

Bosch's data support both the ×8 and ×16 operating points as much more appropriate bandwidth/noise compromises for our purpose than the 0.08 Pa / 12 Hz highest-resolution mode. [24]

Tuning method

Do not manually adjust parameters until it “sounds good” as the first step.

Collect one recording and replay **every estimator against exactly the same data**.

For each configuration sweep:

$$R_h$$

$$q_a$$

$$q_b$$

plus complementary-filter crossover and audio thresholds.

Then evaluate objective metrics.

The parameter set that sounds nicest should only survive if its measured latency/noise metrics also make sense.

Bench tests

T-001 — Stationary noise and timing

Use ESP32-S3 + MS5611 + MPU6050.

Record at least:

```
timestamp
sequence number
raw D1 pressure ADC
raw D2 temperature ADC
compensated pressure
barometric altitude
raw accel XYZ
raw gyro XYZ
attitude quaternion
Earth-Z acceleration
USB enqueue timestamp
USB transmit timestamp
```

Measure:

- pressure RMS and spectrum;
- acceleration RMS;
- bias drift;
- sample-period distribution;
- P50/P95/P99 timing jitter;
- missed samples;
- USB queue depth;
- correlation between multiple MS5611s.

The MS5611's conversion-time dependence on OSR makes this first test directly useful for choosing our acquisition mode. [5]

T-002 — Pressure-only dynamic response

Create controlled pressure ramps corresponding to:

$$0 \rightarrow +0.2\ m/s$$

$$0 \rightarrow +0.5\ m/s$$

$$0 \rightarrow +1.0\ m/s$$

and equally important:

$$+0.2 \rightarrow 0$$

$$+0.5 \rightarrow 0$$

$$+1.0 \rightarrow 0.$$

This can be done with a controlled pressure chamber/syringe fixture before a more sophisticated reference system exists.

This test is **barometric only**. It cannot validate IMU fusion because a pressure ramp does not physically accelerate the instrument.

Measure:

onset latency, **release latency**, **10–90% response time**, overshoot and false oscillation.

T-003 — Real physical motion with synchronized IMU

Move the complete sensor assembly vertically so that pressure and acceleration correspond to the same physical trajectory.

Ideally obtain an independent reference trajectory from a laser/optical measurement, motion-capture system or another suitably precise independent reference.

Replay:

1. raw derivative;

2. conventional low-pass derivative;
3. baro-only 2-state KF;
4. baro-only 3-state `[h,v,a]` KF;
5. complementary baro/IMU estimator;
6. fused `[h,v,b_a]` KF;
7. fast KF + slow confidence bank.

The same raw input must feed all algorithms.

That will answer the most important question in the project without argument or intuition.

T-004 — Artefact and environmental testing

Repeat stationary and controlled-motion tests with:

- direct airflow;
- cross-flow;
- different inlet orientations;
- phone heat;
- ESP32 self-heating;
- touching the dongle;
- cable/connector movement;
- rapid tilt without vertical displacement;
- pitch/roll oscillation;
- sustained USB traffic;
- intentional missed samples;
- one barometer covered/restricted;
- two barometers exposed to common airflow;
- sensors separated physically.

Bosch itself warns that environmental pressure variation can exceed intrinsic sensor noise, making these tests at least as important as electronic-noise measurements. [23]

T-005 — Audio and flight comparison

Instrument the **entire signal-to-sound chain**.

Record simultaneously:

- raw sensor stream;
- estimator output;
- threshold state;
- USB packet/event timestamp;
- app audio state;
- microphone recording of actual audible tone;
- Skytraxx/XC Tracer audio where available.

The crucial metrics are not merely “which one beeps first?”

They are:

Metric	Definition
Onset latency	Reference lift transition → first correct audible lift cue
Release latency	Reference end-of-lift → actual audible lift cue stops
Strong-lift onset	Same metric at +1/+2 m/s
Weak-lift onset	Same metric around +0.1–0.3 m/s
False beep rate	Spurious lift events per minute
Audio tail	Time estimator says stop → sound actually stops
10–90% estimator rise time	Dynamic response independent of threshold
90–10% estimator fall time	Exit dynamic response
Overshoot	Peak error following transition
Stationary v_z RMS	Baseline estimator noise
P95 / P99 sample age	Measurement freshness entering estimator
Dropout rate	Missing acquisition or USB packets
Innovation RMS/NIS	Consistency of Kalman model
IMU–baro disagreement	Dynamic consistency metric
Dual-baro correlation	Separates common environmental from independent noise

A product-level latency target

I would make the first engineering target—not a claimed current capability:

strong-lift onset: <100 ms
weak-lift onset: <150 ms
release: <100–150 ms
estimator-decision-to-audio-stop: <20–30 ms

The last requirement is particularly important.

If the estimator reacts in 80 ms but the beep lasts another 400 ms because its cadence cannot be interrupted, the instrument should **fail our release test**.

What to do about weak lift

Weak lift is the hardest case because the signal amplitude approaches both pressure/environmental noise and normal glider motion.

Do not solve this with a huge smoothing time constant.

Instead use:

small hysteresis + short evidence accumulation + estimator uncertainty + IMU/baro consistency.

For example, rather than saying:

> “Average 500 ms of vertical speed before beeping,”

use:

> “The fast estimate exceeds +0.20 m/s for 40–80 ms and its covariance/innovation is plausible.”

That uses approximately one-tenth the history.

Strong evidence can trigger faster.

Recommended estimator hierarchy

The final prioritisation is:

Best approach

> **Attitude estimator → gravity-compensated vertical acceleration → three-state linear KF `[h,v,b_a]` → parallel slow baro validation KF → confidence-aware, interruptible audio.**

This is the strongest combination of the academic evidence, open-source implementation experience and the very specific failure mode we are trying to improve. [44]

Fallback if the MPU6050/IMU path proves troublesome

> **Two-state barometer KF `[h,v]` with BMP581/585 around ×8 or ×16 OSR, sensor IIR disabled, carefully measured hysteresis and interruptible audio.**

This can still make a very good pressure-only vario, and it provides an indispensable control against which to prove that IMU fusion actually helps.

Do not use as fallback

> “Increase smoothing until the noise disappears.”

That would optimise away the feature we are trying to build.

Short implementation checklist

- ESP32-S3 + ESP-IDF as the acquisition platform. [45]
- MPU6050 at 200 Hz initially.
- MS5611 at 50 Hz/OSR4096 first; test OSR2048 separately. [19]
- Hardware/monotonic timestamps for every sample.
- Store raw pressure, temperature, accel and gyro—not merely filtered values.
- Implement baro-only `[h,v]` KF first.
- Implement complementary baro/IMU filter second.

- Implement primary `[h,v,b_a]` fused KF third.
- Run IMU prediction asynchronously from barometer correction.
- Add slow validation filter without averaging it into `v_fast`.
- Log innovation, covariance and sample age.
- Make lift `STOP` interrupt the currently playing tone.
- Start around +0.20 m/s on / +0.05 m/s off for comparison with established implementations, then tune from our data rather than preference. [27]
- Measure onset **and release** for every filter configuration.
- Upgrade to BMP581/BMP585 only after the acquisition/replay infrastructure works.
- For BMP58x start with ×8 and ×16; bypass its IIR. [46]
- Test dual barometers for noise, common-mode airflow and fault detection—but treat **IMU fusion as the primary latency experiment**.
- Do not move to UKF or complex adaptive banks until recorded-data replay demonstrates a deficiency in the linear fused KF.

The engineering goal can now be stated very precisely:

> **Do not build the smoothest variometer. Build the estimator that gives the earliest *trustworthy* indication of a change in vertical air-mass response—and withdraws that indication just as quickly when the evidence disappears.**

That second half, **withdrawing the cue**, is where both the BlueFly documentation and pilot observations suggest there is unusually valuable room for improvement. [47]

---

[1] [7] [10] [20] [23] [39] BST-BMP585-DS003
https://www.bosch-sensortec.com/media/boschsensortec/downloads/datasheets/bst-bmp585-ds003.pdf

[2] [14] [17] [44] A Sensor Fusion Method for Tracking Vertical Velocity and Height Based on Inertial and Barometric Altimeter Measurements
https://www.mdpi.com/1424-8220/14/8/13324?utm_source=chatgpt.com

[3] [15] IMU-바로미터 기반의 수직변위 추정용 이단계 칼만/상보 필터
https://www.kci.go.kr/kciportal/ci/sereArticleSearch/ciSereArtiView.kci?sereArticleSearchBean.artiId=ART002111843&utm_source=chatgpt.com

[4] Measurement data fusion with cascaded Kalman and complementary filter in the flight parameter indicator for hang-glider and paraglider - ScienceDirect
https://www.sciencedirect.com/science/article/pii/S0263224118300976?utm_source=chatgpt.com

[5] [19] MS5611-01BA03 Barometric Pressure Sensor, with stainless steel cap
https://www.te.com/commerce/DocumentDelivery/DDEController?Action=srchrtrv&DocFormat=pdf&DocLang=English&DocNm=MS5611-01BA03&DocType=Data+Sheet&PartCntxt=MS561101BA03-50

[6] [38] BST-BMP581-DS004-13
https://www.bosch-sensortec.com/media/boschsensortec/downloads/datasheets/bst-bmp581-ds004.pdf?utm_source=chatgpt.com

[8] [12] [25] [26] [27] [30] [35] [42] [47] blueflyvario.com
https://blueflyvario.com/files/BFV_HardwareSettings_Manual_v2.0.pdf

[9] [32] XC Tracer - Gleitschirm und Drachen Forum
https://www.gleitschirmdrachenforum.de/forum/gleitschirm-und-drachen-forum/technik/32244-xc-tracer?utm_source=chatgpt.com

11 Kalman15
https://www.cs.yale.edu/homes/lans/readings/general/Kalman1960.pdf?utm_source=chatgpt.com

13 29 GitHub - har-in-air/ESP32_IMU_BARO_GPS_VARIO: GPS altimeter/variometer with LCD display, routes with waypoints, data/gps track logging, bluetooth NMEA sentence transmission, wifi AP + webpage configuration · GitHub
https://github.com/har-in-air/ESP32_IMU_BARO_GPS_VARIO?utm_source=chatgpt.com

16 Unscented filtering and nonlinear estimation | IEEE Journals & Magazine | IEEE Xplore
https://ieeexplore.ieee.org/abstract/document/1271397?utm_source=chatgpt.com

18 Radar-Altimeter Inertial Vertical Loop—Multisensor Estimation of Vertical Parameters for Autonomous Vertical Landing
https://www.mdpi.com/2673-4591/88/1/28?utm_source=chatgpt.com

21 41 Global Search | TDK InvenSense
https://www.invensense.tdk.com/en-us/search-result?query=6050&utm_source=chatgpt.com

22 45 General Purpose Timer (GPTimer) - ESP32-S3 - — ESP-IDF Programming Guide v6.1 documentation
https://docs.espressif.com/projects/esp-idf/en/stable/esp32s3/api-reference/peripherals/gptimer.html?utm_source=chatgpt.com

24 46 BST-BMP585-DS003
https://www.bosch-sensortec.com/media/boschsensortec/downloads/datasheets/bst-bmp585-ds003.pdf?utm_source=chatgpt.com

28 USB Device Stack - ESP32-S3 - — ESP-USB Programming Guide latest documentation
https://docs.espressif.com/projects/esp-usb/en/latest/esp32s3/usb_device.html?utm_source=chatgpt.com

31 XC Tracer Mini V | XC Tracer
https://www.xctracer.com/en/product-page/xc-tracer-mini-v?utm_source=chatgpt.com

33 Minivarios...Übersicht & praktische Erfahrungen? - Gleitschirm und Drachen Forum
https://www.gleitschirmdrachenforum.de/forum/gleitschirm-und-drachen-forum/technik/35933-minivarios-%C3%BCbersicht-praktische-erfahrungen?utm_source=chatgpt.com

34 Downloads | XC Tracer Mini V
https://www.xctracer.com/downloadsxctracerminiv?utm_source=chatgpt.com

36 GitHub - har-in-air/ESP32_IMU_BARO_GPS_LOGGER: ESP32 high-speed flash data logger for IMU (accelerometer+gyro+magnetometer), GPS, pressure sensor readings. · GitHub
https://github.com/har-in-air/ESP32_IMU_BARO_GPS_LOGGER?utm_source=chatgpt.com

37 GitHub - juangallostra/AltitudeEstimation: A two-step Kalman/Complementary filter for Estimation of Vertical Position using an IMU-Barometer sytem · GitHub
https://github.com/juangallostra/AltitudeEstimation?utm_source=chatgpt.com

40 MS561101BA03-50 : MEAS Board Mount Pressure Sensors | TE Connectivity
https://www.te.com/en/product-MS561101BA03-50.html

43 XC Tracer Mini V | XC Tracer
https://www.xctracer.com/en/xctracerminiiiigps?utm_source=chatgpt.com